ALGORITHM DESIGN 'S TERM PROJECT

Presented by

Thit Lwin Win Thant – 6540122

Kaung Khant Lin – 6540131

PROBLEM SOURCE

Source: Google Code Jam '22 Round 1A Problem C – Weightlifting from dmoj online judge website

Link: Google Code Jam '22 Round 1A Problem C – Weightlifting

PROBLEM DESCRIPTION

- a vertical exercise machine
- Load weight plates onto the top
- remove them from the top
- Adding or removing a single plate costs 1 operation
- The machine starts empty and must end empty
- The core rule: load the required weights for an exercise in any order

OUR GOAL

Our goal is to find the minimum total number of operations (pushes + pops) required to complete all exercises and leave the stack empty at the end.

INPUT SPECIFICATION

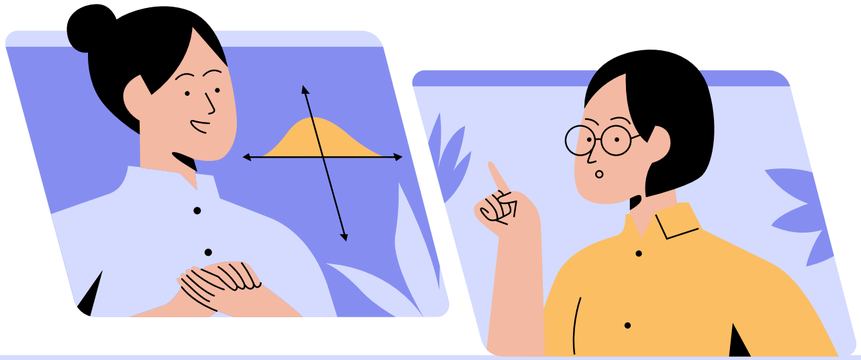
- Number of Workouts: the number of total workout sessions (Test Cases)

- Workout Details: For each workout, it gets two numbers:
  - total exercises we have to do
  - different types of weights the gym has.

- The Routine: a grid of numbers
  - each row represents one exercise
  - the numbers tell us exactly how many of each weight type we need on the machine to do that exercise

OUTPUT SPECIFICATION

For each test case, output one line containing Case #x: y, where x is the test case number (starting from 1) and y is the minimum number of machine stack operations needed to run through all our exercises.



CONSTRAINTS AND LIMITS

- Time limit: 20 seconds

- Memory limit: 1 GB

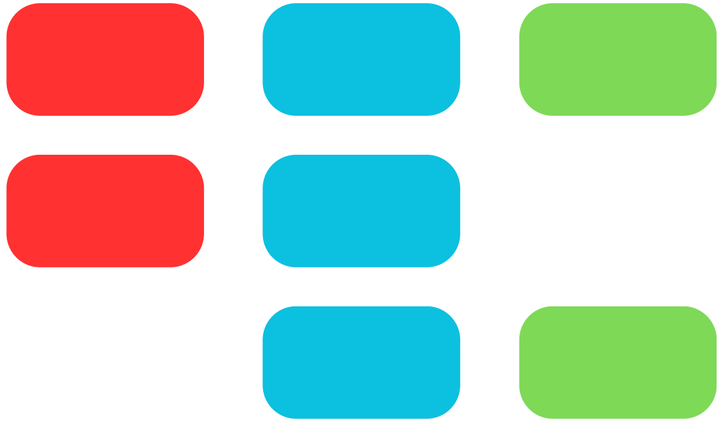
- $1 \leqslant T \leqslant 100$. (Maximum Test Cases = 100)
- Each exercise requires at least one weight

- Test Set 1
  - $1 \leqslant E \leqslant 10$.
  - $1 \leqslant W \leqslant 3$.
  - $0 \leqslant X_{i,j} \leqslant 3$, for all i, j.

- Test Set 2
  - $1 \leqslant E \leqslant 100$.
  - $1 \leqslant W \leqslant 100$.
  - $0 \leqslant X_{i,j} \leqslant 100$, for all i, j.

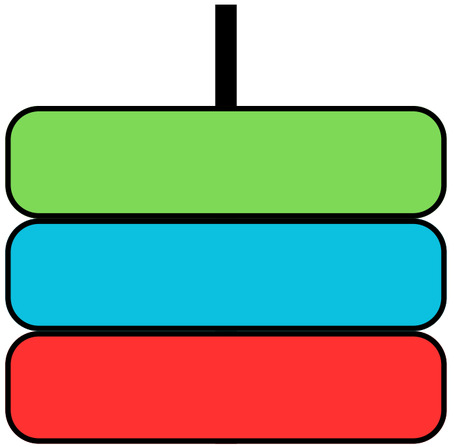
EXAMPLE TEST CASE

- 3-exercise workout using Red, Blue, and Green plates.
- Exercise 1: Needs
- Exercise 2: Needs
- Exercise 3: Needs



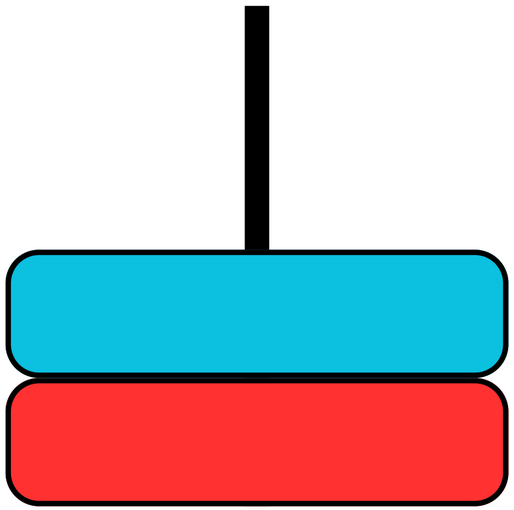
THE TRAP (WHY NORMAL HUMAN LOGIC / GREEDY FAILS)

- Just looking one step ahead – fall into a trap
- At Ex 1, look ahead to Exercise 2:
  - both need a and a
  - decide to put those at the bottom to save time
  - stack the machine from bottom to top: =>
  - do Exercise 1. (Cost to build: 3 pushes)



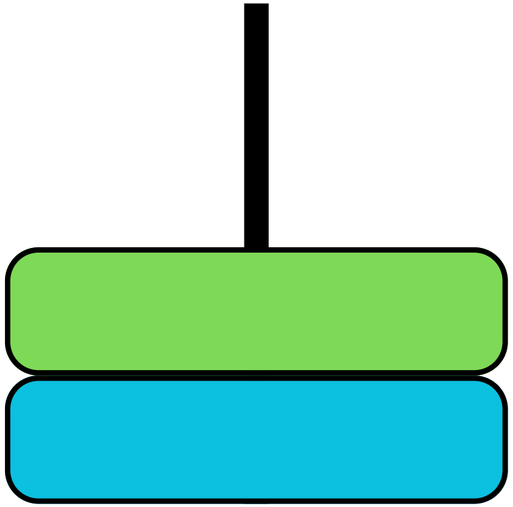
THE TRAP (WHY NORMAL HUMAN LOGIC / GREEDY FAILS)

- Move to Exercise 2:
  - needs Red and Blue
  - Cost: 1 remove (pop) for Green
- Move to Exercise 3:
  - needs a Blue and a Green.
  - Look at our stack:
  - Reason: only looked one step ahead
  - forced to tear the stack down
  - must pop Blue, pop the Red in order to remove Red



THE TRAP (WHY NORMAL HUMAN LOGIC / GREEDY FAILS)

- Move to Exercise 3:
  - Need to put back Blue
  - Then Green
  - Cost: 4 operations
- Total operations for this bad strategy : 10

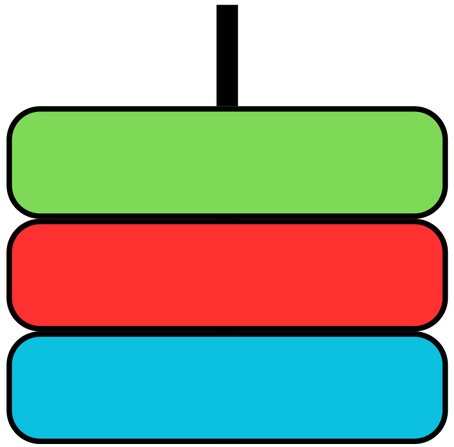


THE CORE MECHANIC (THE GLOBAL VIEW)

- To minimize operations: must find the plates that are shared across the entire sequence and anchor them at the absolute bottom
- Look at Exercise 1, 2, and 3 all together: Global Shared Plate = "Blue"

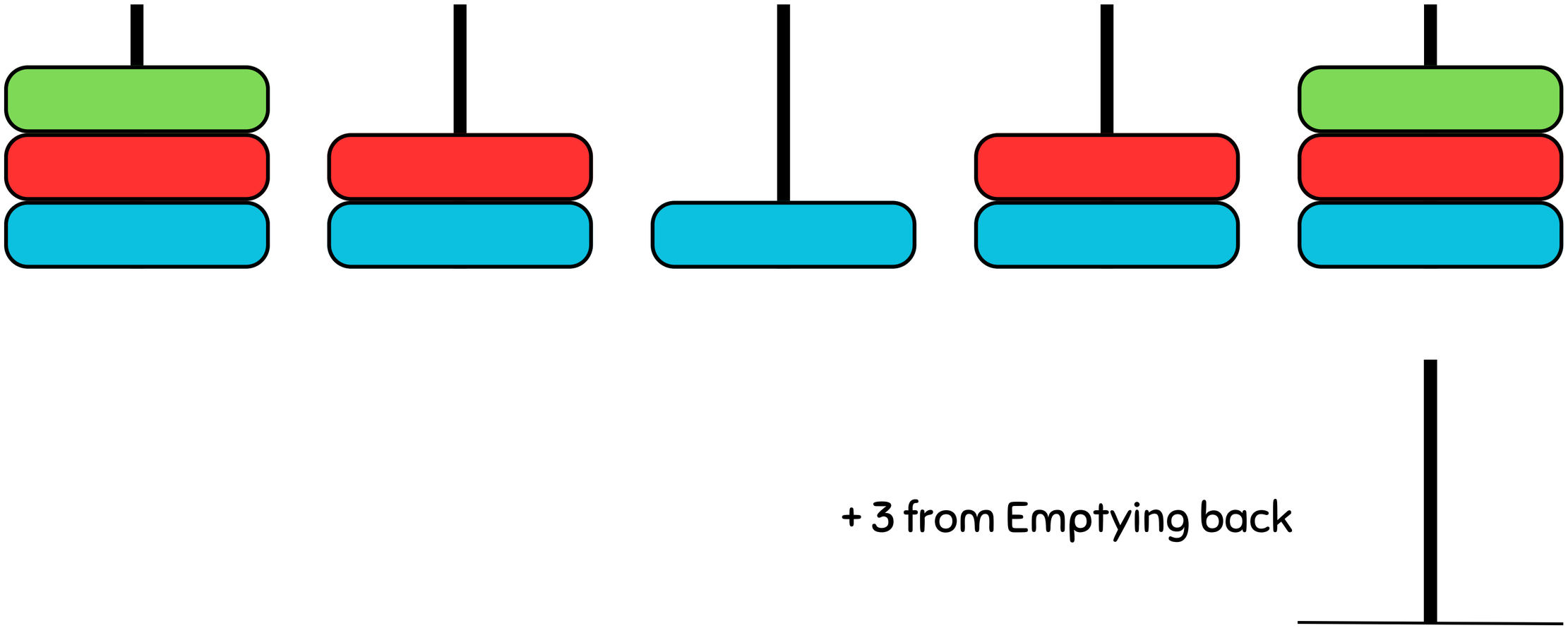
- Exercise 1: Needs
- Exercise 2: Needs
- Exercise 3: Needs

THE CORE MECHANIC (THE GLOBAL VIEW)



- Knowing Stack from bottom to top: [Blue, Red, Green].
- Do Exercise 1. (Cost: 3 pushes).
- Move to Exercise 2 (needs Red, Blue).
- pop the Green. Stack is [Blue, Red]. (Cost: 1 pop).
- Move to Exercise 3 (needs Blue, Green).
- Pop the Red, push the Green. Stack is [Blue, Green]. (Cost: 2 ops).
- Workout over. Empty the machine. (Cost: 2 pops). (Optimal = 8 Operations)

THE CORE MECHANIC (THE GLOBAL VIEW)

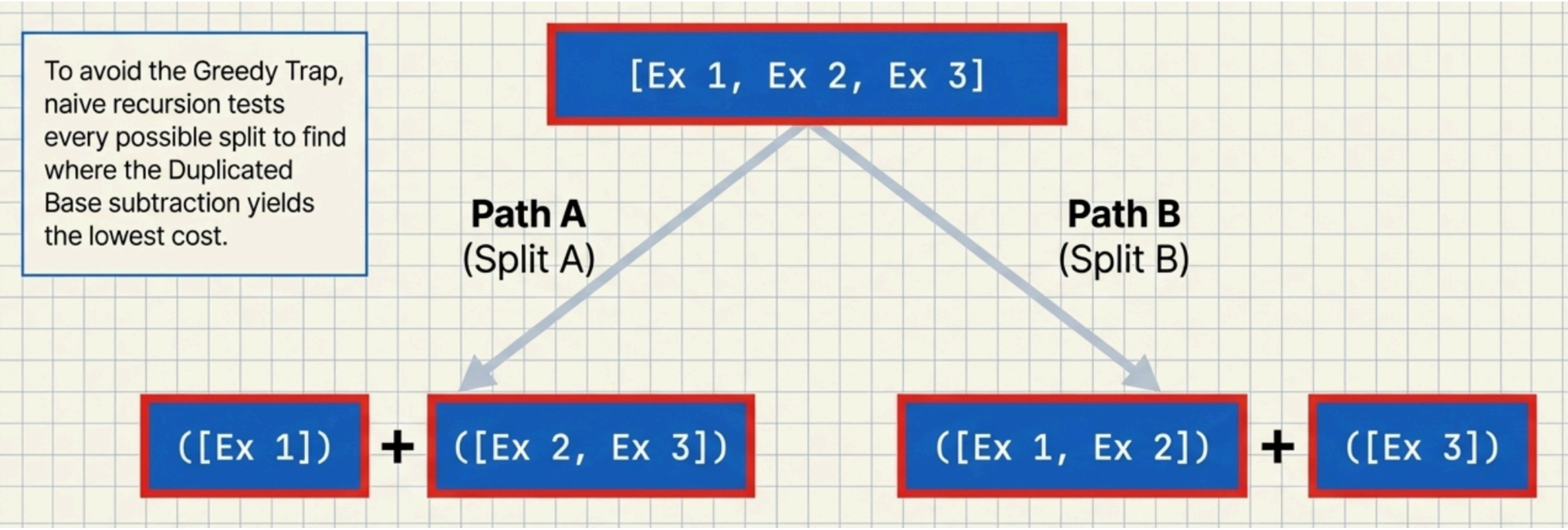


THE CORE MECHANIC (THE GLOBAL VIEW)

```
# Total Cost = Left + Right - Duplicated Global Base
total_cost = cost_left + cost_right - savings
```

- Example: If a "Blue plate" spans both merged groups, we subtract the 2 redundant operations (the duplicate push and pop) to get the true optimal cost.

NAIVE RECURSION WALKTHROUGH



TOP-DOWN DP (MEMOIZATION)

- Introducing a Cache: to store the results of previously computed intervals to prevent recursive explosion.
- Save and Reuse: Once a specific sub-interval (e.g., [Ex 2, Ex 3]) is evaluated for the first time, its optimal cost is saved.
- Instant Retrieval: If a different branch of the algorithm later asks for that exact same interval, it instantly fetches the cached value instead of recalculating the math.

THE TRANSITION TO INTERVAL DP

- Step 1 (Base Cases): Calculating single exercises yields 6 operations for dp[0, 0], 4 operations for dp[1, 1], and 4 operations for dp[2, 2].

dp[i][j]	j = 0	j = 1	j = 2
i = 0	6	-	-
i = 1	-	4	-
i = 2	-	-	4

THE TRANSITION TO INTERVAL DP

- Step 2 (Length 2 Intervals): Merging Ex 1 and Ex 2 (dp[0, 1])
- Merging Ex 2 and Ex 3 (dp[1, 2])

dp[i][j]	j = 0	j = 1	j = 2
i = 0	6	6	-
i = 1	-	4	6
i = 2	-	-	4

- Red and Blue weights for a savings of 4, costing 6 operations total (6 + 4 – 4)
- Blue weight for a savings of 2, costing 6 operations total (4 + 4 – 2).

THE TRANSITION TO INTERVAL DP

- Step 3 (Length 3 Sequence): The algorithm evaluates all splits for the complete sequence, applying a savings of 2 for the globally shared Blue weight.

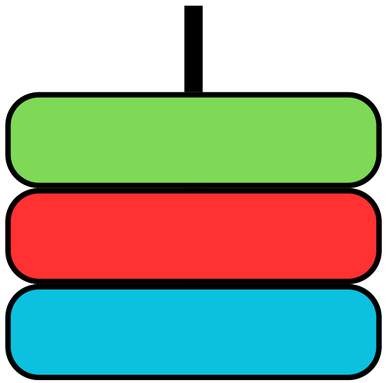
dp[0][2]

Split after Ex1	Split after Ex2
dp[0][0] + dp[1][2]	dp[0][1] + dp[2][2]
6 + 6 − 2	6 + 4 − 2
= 10 operations	= 8 operations ✓ <u>Optimal</u>

THE TRANSITION TO INTERVAL DP

- Evaluating Splits: Splitting after Ex 1 calculates to 10 operations (6 + 6 – 2), while splitting after Ex 2 calculates to the optimal 8 operations (6 + 4 – 2).

- The Physical Meaning: The final output of 8 operations represents physically anchoring the shared Blue weight at the absolute bottom of the stack to avoid forced teardowns.



TIME AND SPACE COMPLEXITY

Algorithm	Time Complexity	Space Complexity
Naive Recursion	$O(4^E / E^{\frac{3}{2}})$	O(E)
Memoization (Top-Down DP)	$O(E^3 \times W)$	$O(E^2)$
Interval DP (Bottom-Up)	$O(E^3 + E^2 W)$	$O(E^2)$

- E : Number of exercises
- W : Number of distinct weight types

THANK YOU